

浮点数的表示方法

$$9 \times 10^{-28} = 0.9 \times 10^{-27}$$

$$2 \times 10^{33} = 0.2 \times 10^{34}$$

$$\text{—— } N = 10^E \cdot M \quad (\text{十进制表示})$$

任意一个十进制数 N 可以写成

$$N = 10^E \times M$$

计算机中一个任意进制数 N 可以写成

$$N = R^e \times m$$

m : 尾数, 是一个纯小数。

e : 浮点的指数, 是一个整数。

R : 基数, 对于二进计数值的机器是一个常数, 一般规定

R 为2, 8或16。

一个机器浮点数由阶码和尾数及其符号位组成：

尾数：用定点小数表示，给出有效数字的位数，
决定了浮点数的表示精度；

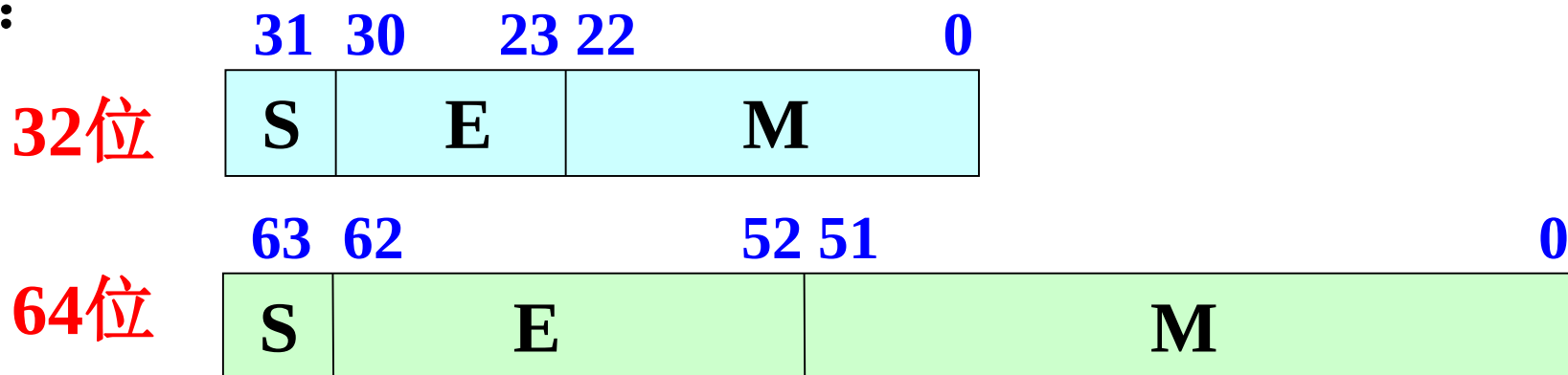
阶码：用定点整数形式表示，指明小数点在数据中的位置，
决定了浮点数的表示范围。

E_s	$E_1 E_2 \dots E_m$	M_s	$M_1 M_2 \dots M_n$
阶符	阶码	数符	尾数

(2) 浮点数的标准格式 $(N = R^e.m)$

为便于软件移植，使用 IEEE（电气和电子工程师协会）标准
IEEE754 标准：尾数用原码；阶码用移码；基为2

• 按照 IEEE754 的标准，32位浮点数和64位浮点数的标准格式为：



S——尾数符号，0正1负；

M——尾数，纯小数表示，小数点放在尾数域的最前面。
采用原码表示。

E——阶码，采用“**移码**”表示；

阶符采用隐含方式，即采用“**移码**”方法来表示正负指数。

(3) 浮点数的规格化表示

浮点数是数学中实数的子集合，由一个纯小数乘上一个指数值来组成。

一个浮点数有不同的表示：

0.5 ； 0.05×10^1 ； 0.005×10^2 ； 50×10^{-2}

为提高数据的表示精度，需做规格化处理。

浮点数的规格化

规格化目的:

为了提高数据的表示精度

为了数据表示的唯一性

尾数为R进制的规格化:

绝对值大于或等于 $1/R$

二进制原码的规格化数的表现形式:

正数 0.1xxxxxx

负数 1.1xxxxxx

补码尾数的规格化的表现形式:

尾数的最高位与符号位相反

正数 0.1xxxxxx

负数 1.0xxxxxx

规格化处理:

在计算机内, 其纯小数部分被称为浮点数的尾数, 对非 0 值的浮点数, 要求尾数的绝对值**必须 $\geq 1/2$** , 即尾数域的最高有效位应为1, 称满足这种表示要求的浮点数为**规格化表示**:

0. 1000101010

把不满足这一表示要求的尾数, 变成满足这一要求的尾数的操作过程, 叫作浮点数的**规格化处理**, 通过**尾数移位和修改阶码实现**。

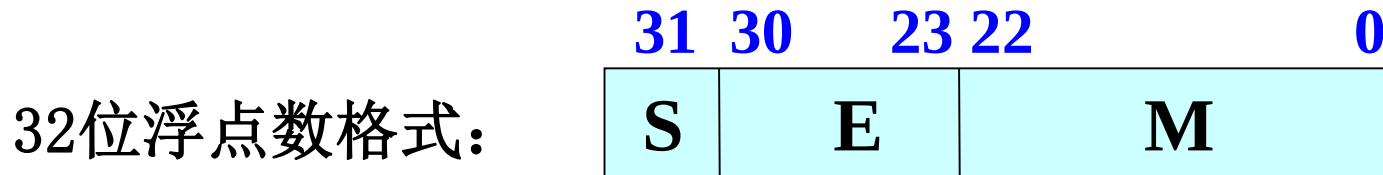
隐藏位技术:

既然非 0 值浮点数的尾数数值最高位必定为 1，则在保存浮点数到内存前，通过尾数左移，强行把该位去掉，用同样多的尾数位就能多存一位二进制数，有利于提高数据表示精度，称这种处理方案使用了**隐藏位**技术。

$$0.1100010 \rightarrow 1.100010$$

当然，在取回这样的浮点数到运算器执行运算时，必须先恢复该隐藏位。

(4) 规格化浮点数的真值



移码定义: $[x]_{\text{移}} = x_0 x_1 x_2 \cdots x_n$
 $= 2^n + x$ $-2^n \leq x < 2^n$

一个规格化的32位浮点数 x 的真值为:

$$x = (-1)^s \times (1.M) \times 2^{E-127} \quad e = E - 127$$

一个规格化的64位浮点数 x 的真值为:

$$x = (-1)^s \times (1.M) \times 2^{E-1023}$$

这里 e 是真值, E 是机器数

例：若浮点数 x 的二进制存储格式为 $(41360000)_{16}$ ，求其32位浮点数的十进制值。

解： **0**100,0001,0011,0110,0000,0000,0000,0000

数符： 0

阶码： 1000,0010

尾数： 011,0110,0000,0000,0000,0000

指数 $e = \text{阶码} - 127 = 10000010 - 01111111 = 00000011 = (3)_{10}$

包括隐藏位1的尾数：

$1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$

于是有 $x = (-1)^s \times 1.M \times 2^e$

$= + (1.011011) \times 2^3 = + 1011.011 = (11.375)_{10}$

例：将十进制数20.59375转换成32位浮点数的二进制格式来存储

解：首先分别将整数和分数部分转换成二进制数：

$$20.59375 = 10100.10011$$

然后移动小数点，使其在第1，2位之间

$$10100.10011 = 1.010010011 \times 2^4 \quad e=4$$

于是得到： $e = E - 127$

$$S=0, \quad E=4+127=131=1000,0011, \quad M=010010011$$

最后得到32位浮点数的二进制存储格式为

$$0100 \ 0001 \ 1010 \ 0100 \ 1100 \ 0000 \ 0000 \ 0000 = (41A4C000)_{16}$$

例：将十进制数-0.75表示成单精度的IEEE 754标准代码

解： $-0.75 = -3/4 = -0.11_2 = -1.1 \times 2^{-1}$

$$=(-1)^1 \times (1 + 0.1000\ 0000\ 0000\ 0000\ 0000\ 000) \times 2^{-1}$$

$$=(-1)^1 \times (1 + 0.1000\ 0000\ 0000\ 0000\ 0000\ 000) \times 2^{126-127}$$

$$s=1, E=126_{10}=01111110_2, F=1000\ \dots\ 000。$$

1 011, 1111, 0 100, 0000, 0000, 0000, 0000, 0000
B F 4 0 0 0 0 0 H

例：求如下IEEE 754 单精度浮点数的十进制数值：

1 10000001 010000000000000000000000

解： $S=1$, $E=129$, $F = 1/4 = 0.25$,

$$(-1)^1 \times (1+0.25) \times 2^{129-127}$$

$$= -1 \times 1.25 \times 2^2$$

$$= -1.25 \times 4$$

$$= -5.0$$

例：对数据 123_{10} 作规格化浮点数的编码，假定1位符号位，基数为2，阶码5位，采用移码，尾数10位，采用补码。

解： $123_{10}=1111011_2= 0.1111011000_2 \times 2^7$

$$[7]_{\text{移}}=10000+00111 = 10111$$

$$[0.1111011000]_{\text{补}}=0.1111011000$$

$$[123]_{\text{浮}}=0 \ 10111 \ 1111011000$$

IEEE754浮点数的范围

格式	最小值	最大值
单精度	$E=1, M=0,$ $1.0 \times 2^{1-127} = 2^{-126}$	$E=254, f=.1111 \dots,$ $1.111 \dots 1 \times 2^{254-127}$ $= 2^{127} \times (2-2^{-23})$
双精度	$E=1, M=0,$ $1.0 \times 2^{1-1023} = 2^{-1022}$	$E=2046, f=.1111 \dots,$ $1.111 \dots 1 \times 2^{2046-1023}$ $= 2^{1023} \times (2-2^{-52})$

浮点数标准 (IEEE754)

IEEE 单精度浮点数编码格式

符号位	阶码	尾数	表示
0/1	255	非 0	NaN
0/1	255	非 0	NaN
0	255	0	+INF
1	255	0	-INF
0/1	1-254	f	$(-1)^s \times 1.f \times 2^{e-127}$
0/1	0	$f(\text{非 } 0)$	$(-1)^s \times 0.f \times 2^{-126}$
0/1	0	0	+0/-0

浮点运算方法和浮点运算器

浮点加、减法运算

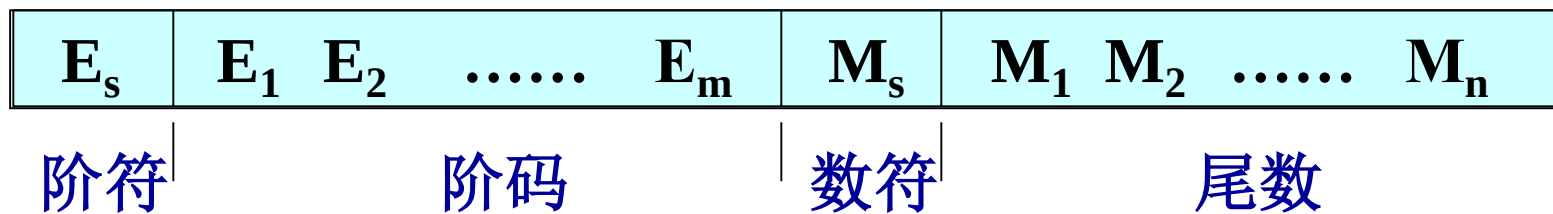
浮点乘、除法运算

浮点数的表示方法

机器浮点数格式:

尾数: 用定点小数表示, 给出有效数字的位数, 决定了浮点数的表示精度;

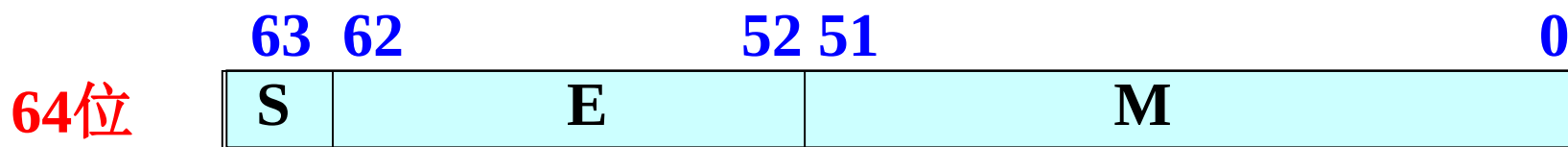
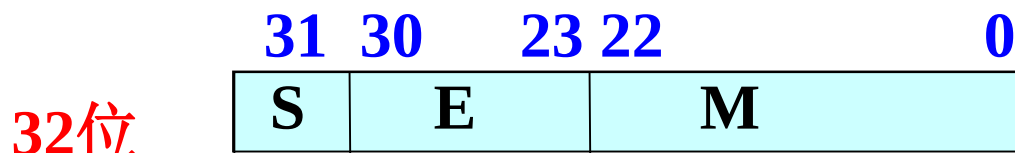
阶码: 用整数形式表示, 指明小数点在数据中的位置, 决定了浮点数的表示范围。



浮点数的标准格式

- 为便于软件移植，使用 IEEE标准

按照 IEEE754 的标准，32位浮点数和64位浮点数的标准格式为：



IEEE 标准：尾数用原码；阶码用“移码”；基为2。

浮点加、减法运算

设有两个浮点数 x 和 y , 它们分别为:

$$x = 2^{E_x} \cdot M_x$$

$$y = 2^{E_y} \cdot M_y$$

其中 E_x 和 E_y 分别为数 x 和 y 的阶码,

M_x 和 M_y 为数 x 和 y 的尾数。

两浮点数进行加法和减法的运算规则是:

$$x \pm y = (M_x 2^{E_x - E_y} \pm M_y) 2^{E_y} \quad E_x \leq E_y$$

完成浮点加减运算的操作过程大体分为四步：

- (1) 0 操作数的检查；
- (2) 比较阶码大小并完成对阶；
- (3) 尾数进行加或减运算；
- (4) 结果规格化。
- (5) 舍入处理。
- (6) 溢出处理。

(1) 0 操作数检查

(2) 对阶

使二数阶码相同（即小数点位置对齐），这个过程叫作**对阶**。

- 先求两数阶码 E_x 和 E_y 之差，即 $\Delta E = E_x - E_y$

若 $\Delta E = 0$ ，表示 $E_x = E_y$

若 $\Delta E > 0$ ， $E_x > E_y$

若 $\Delta E < 0$ ， $E_x < E_y$

通过尾数的移动来改变 E_x 或 E_y ，使其相等。

- 对阶原则**

阶码小的数向阶码大的数对齐；

小阶的尾数右移，每右移一位，其阶码加1（**右规**）。

例： $x=2^{01} \times 0.1101$, $y=2^{11} \times (-0.1010)$, 求 $x+y=?$

解： 为便于直观了解，两数均以补码表示，阶码、尾数均采用双符号位。

$$[x]_{\text{补}} = 00\ 01, \ 00.1101 \quad [y]_{\text{补}} = 00\ 11, \ 11.0110$$

$$[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00\ 01 + 11\ 01 = 11\ 10$$

$\Delta E = -2$, 表示 E_x 比 E_y 小 2, 因此将 x 的尾数右移两位.

右移一位, 得 $[x]_{\text{补}} = 00\ 10, \ 00.0110$

再右移一位, 得 $[x]_{\text{补}} = 00\ 11, \ 00.0011$

至此, $\Delta E = 0$, 对阶完毕.

(3) 尾数求和运算

尾数求和方法与定点加减法运算完全一样。

对阶完毕可得： $[x]_{\text{补}} = 00\ 11, 00.0011$

$[y]_{\text{补}} = 00\ 11, 11.0110$

对尾数求和：

$$\begin{array}{r} 00.0011 \\ + 11.0110 \\ \hline 11.1001 \end{array}$$

即得： $[x+y]_{\text{补}} = 00\ 11, 11.1001$

(4) 结果规格化

求和之后得到的数可能不是规格化了的数，为了增加有效数字的位数，提高运算精度，必须将求和的结果规格化。

①规格化的定义： $\frac{1}{2} \leq |S| < 1$ (二进制)

采用原码：

正数： $S=0.1 \times \times \times \dots \times$

负数： $S=1.1 \times \times \times \dots \times$

采用双符号位的补码：

对正数： $S=00.1 \times \times \times \dots \times$

对负数： $S=11.0 \times \times \times \dots \times$

②向左规格化

若不是规格化的数, 需要尾数**向左**移位, 以实现规格化的过程, 我们称其为**向左规格化**。

前例中, **00 11, 11. 1001**不是规格化数, 因而需要左规, 即**左移一位, 阶码减1**, 得:

$$[x+y]_{\text{补}} = 00\ 10, 11.0010$$

③向右规格化

浮点加减运算时, 尾数求和的结果也可能得到:

01. ×××...× 或 10. ×××...×,

即两符号位不等, 即结果的绝对值大于1。 **向左破坏了规格化**。

此时, 将尾数运算的结果右移一位, 阶码加1, 称为**向右规格化**。

例：两浮点数 $x=0.1101 \times 2^{10}$, $y=(0.1011) \times 2^{01}$, 求 $x+y$ 。

解： $[x]_{\text{补}}=00\ 10, 00.1101$ $[y]_{\text{补}}=00\ 01, 00.1011$

对阶： $[\Delta E]_{\text{补}}=[E_x]_{\text{补}}-[E_y]_{\text{补}}=00\ 10+11\ 11=00\ 01$

y 向 x 对齐，将 y 的尾数右移一位，阶码加1。

$[y]_{\text{补}}=00\ 10, 00.0101$

求和：

$$\begin{array}{r} 00.1101 \\ + 00.0101 \\ \hline 01.0010 \end{array}$$

$[x+y]_{\text{补}}=00\ 10, 01.0010$

右归：运算结果两符号位不同，其绝对值大于1，右归。

$[x+y]_{\text{补}}=00\ 11, 00.1001$

(5) 舍入处理

在对阶或向右规格化时,尾数要向右移位,这样,被右移的尾数的低位部分会被丢掉,从而造成一定误差,因此要进行舍入处理。

- 简单的舍入方法有两种:

① “0舍1入”法

即如果右移时被丢掉数位的最高位为0则舍去,反之则将尾数的末位加“1”。

② “恒置1”法

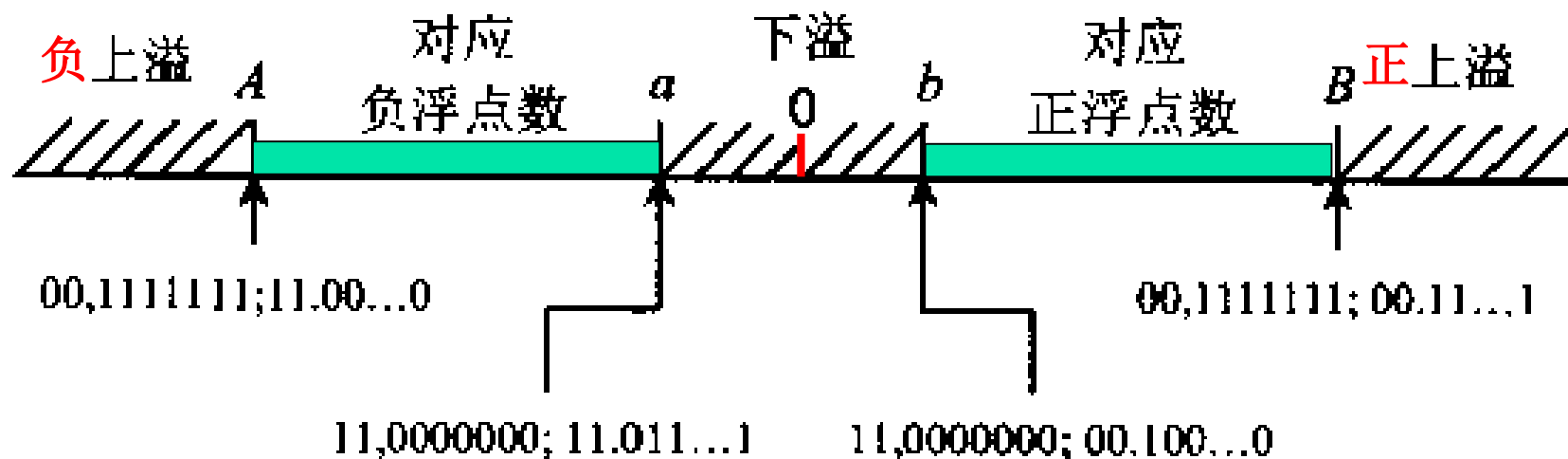
即只要数位被移掉,就在尾数的末位恒置“1”。从概率上来说,丢掉的0和1各为1/2。

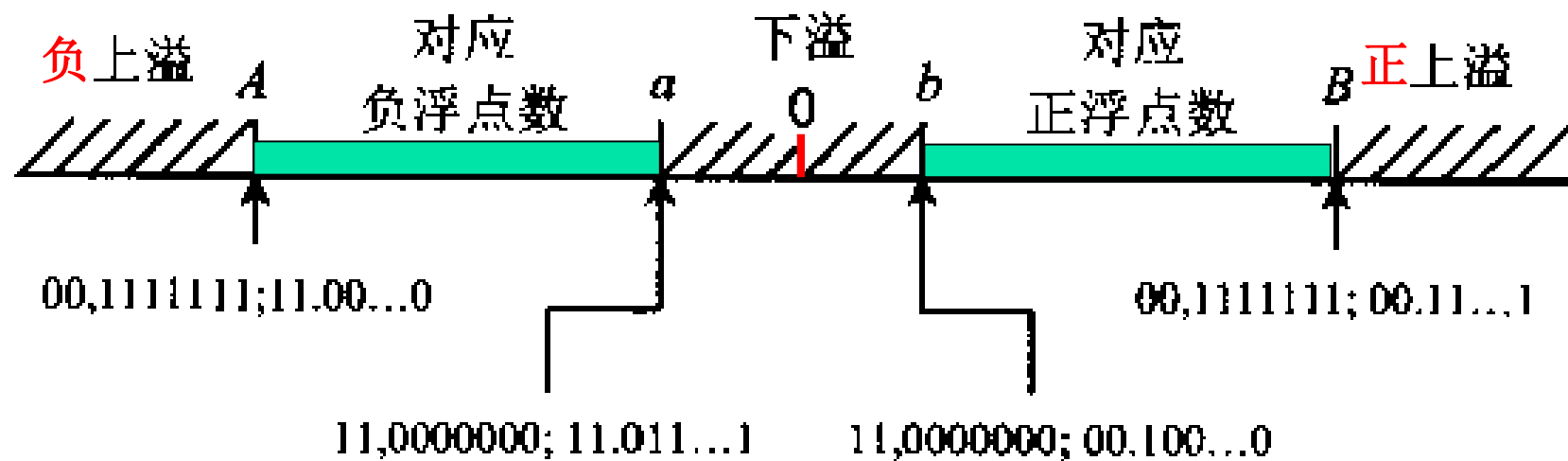
- IEEE754标准中,舍入处理提供了四种可选方法:

(6) 溢出处理

与定点加减法一样，浮点加减运算最后一步也需判溢出。在浮点规格化中已指出，当尾数之和(差)出现 $01.\times\times\dots\times$ 或 $10.\times\times\dots\times$ 时，并不表示溢出，只有将此数右规后，再根据阶码来判断浮点运算结果是否溢出。

若机器数为补码，尾数为规格化形式，并假设阶符取2位，阶码取7位、数符取2位，尾数取 n 位，则它们能表示的补码在数轴上的表示范围如图所示。





图中A，B，a，b分别对应最小负数、最大正数、最大负数和最小正数。它们所对应的真值分别是：

A最小负数 $2^{+127} \times (-1)$

B最大正数 $2^{+127} \times (1-2^{-n})$

a最大负数 $2^{-128} \times (-2^{-1}-2^{-n})$

b最小正数 $2^{-128} \times 2^{-1}$

图中a,b之间的阴影部分，对应阶码小于128的情况，叫做浮点数的下溢。下溢时，浮点数值趋于零，故机器不做溢出处理，仅把它作为机器零。

图中的A、B两侧阴影部分，对应阶码大于127的情况，叫做浮点数的上溢。此刻，浮点数真正溢出，机器需停止运算，作溢出中断处理。一般说浮点溢出，均是指上溢。

可见，浮点机的溢出与否可由阶码的符号决定：

阶码 $[j]_{\text{补}} = 01, \times \times \times \times \times$ 为上溢，机器停止运算，做中断处理；

阶码 $[j]_{\text{补}} = 10, \times \times \times \times \times$ 为下溢，按机器零处理。

例：若某次加法操作的结果为 $[X+Y]_{\text{补}} = 00.111, 10.1011100111$

则应对其进行向右规格化操作：

尾数为： 11.0101110011 ， 阶码加1： 01.000

阶码超出了它所能表示的最大正数（+7），表明本次浮点运算产生了溢出。

例：若某次加法操作的结果为 $[X+Y]_{\text{补}} = 11.010, 00.0000110111$

则应对其进行向左规格化操作：

尾数为： 00.1101110000 ， 阶码减4：

$$\begin{array}{r} 11.010 \\ + 11.100 \\ \hline 10.110 \end{array} \quad [-4]_{\text{补}}$$

阶码超出了它所能表示的最小负数（-8），表明本次浮点运算产生了溢出。

小结:

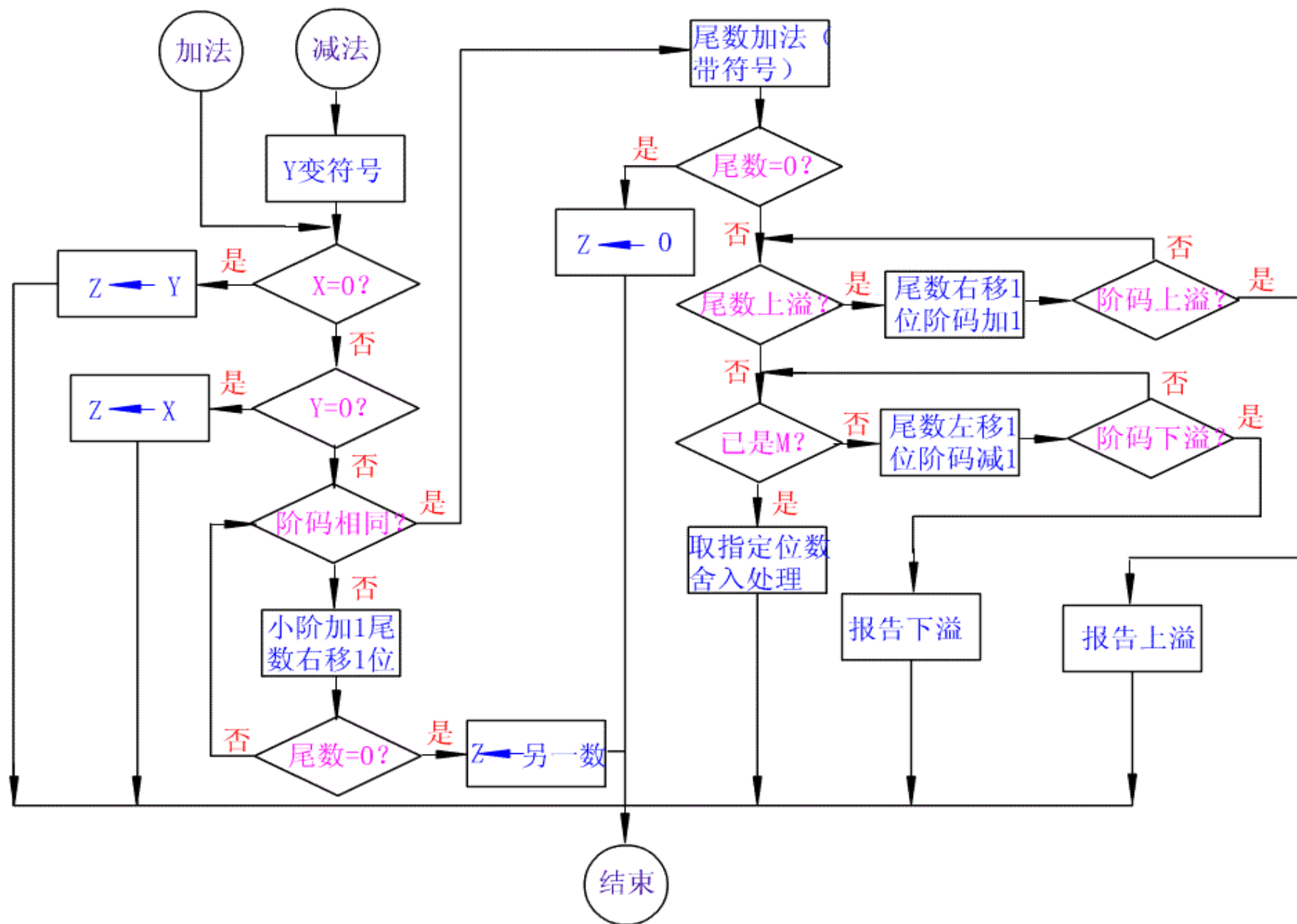
- 浮点数的溢出是以其阶码溢出表现出来的

在加、减运算过程中要检查是否产生了溢出：若阶码正常，加减运算正常结束；若阶码溢出，则要进行相应的处理。

阶码上溢——超过了阶码可能表示的最大值的正指数值,一般将其认为是 $+\infty$ 和 $-\infty$ 。

阶码下溢——超过了阶码可能表示的最小值的负指数值,一般将其认为是0。

- 对尾数的溢出也需要处理(上溢—右归, 下溢—舍入)。



例 设 $x = 2^{010} \times 0.11011011$, $y = 2^{100} \times (-0.10101100)$, 求 $x + y$ 。

解： 阶码采用双符号位, 尾数采用单符号位, 则它们的浮点表示分别为

$$[x]_{\text{浮}} = 00\ 010, \quad 0.11011011$$

$$[y]_{\text{浮}} = 00\ 100, \quad 1.01010100$$

(1) 求阶差并对阶

$$\Delta E = E_x - E_y = [E_x]_{\text{补}} + [-E_y]_{\text{补}} = 00\ 010 + 11\ 100 = 11\ 110$$

即 ΔE 为 -2, x 的阶码小, 应使 M_x 右移两位, E_x 加 2,

$$[x]_{\text{浮}} = 00\ 100, \quad 0.00110110(11)$$

其中(11)表示 M_x 右移 2 位后移出的最低两位数。

(2) 尾数求和

$$\begin{array}{r} 0.00110110(11) \\ + 1.01010100 \\ \hline 1.10001010(11) \end{array}$$

(3) 规格化处理

尾数运算结果的符号位与最高数值位为同值，应执行左规处理，结果为1.00010101(10)，阶码为00 011。

(4) 舍入处理

采用0舍1入法处理，则有：

$$\begin{array}{r} 1.00010101 \\ + \quad \quad \quad 1 \\ \hline 1.00010110 \end{array}$$

(5) 判断溢出

阶码符号位为00，不溢出，故得最终结果为

$$x + y = 2^{011} \times (-0.11101010)$$

例 两浮点数 $x = 2^{01} \times 0.1101$, $y = 2^{11} \times (-0.1010)$ 。假设尾数在计算机中以补码表示, 可存储4位尾数, 2位保护位, 阶码以原码表示, 求 $x+y$ 。

解: 将 x, y 转换成浮点数据格式

$$[x]_{\text{浮}} = 00\ 01, 00.1101$$

$$[y]_{\text{浮}} = 00\ 11, 11.0110$$

步骤1: 对阶, 阶差为 $11-01=10$, 即2, 因此将 x 的尾数右移两位, 得

$$[x]_{\text{浮}} = 00\ 11, 00.001101$$

步骤2: 对尾数求和, 得:

$$[x+y]_{\text{浮}} = 00\ 11, 11.100101$$

步骤3: 由于符号位和第一位数相等, 不是规格化数, 向左规格化, 得

$$[x+y]_{\text{浮}} = 00\ 10, 11.001010$$

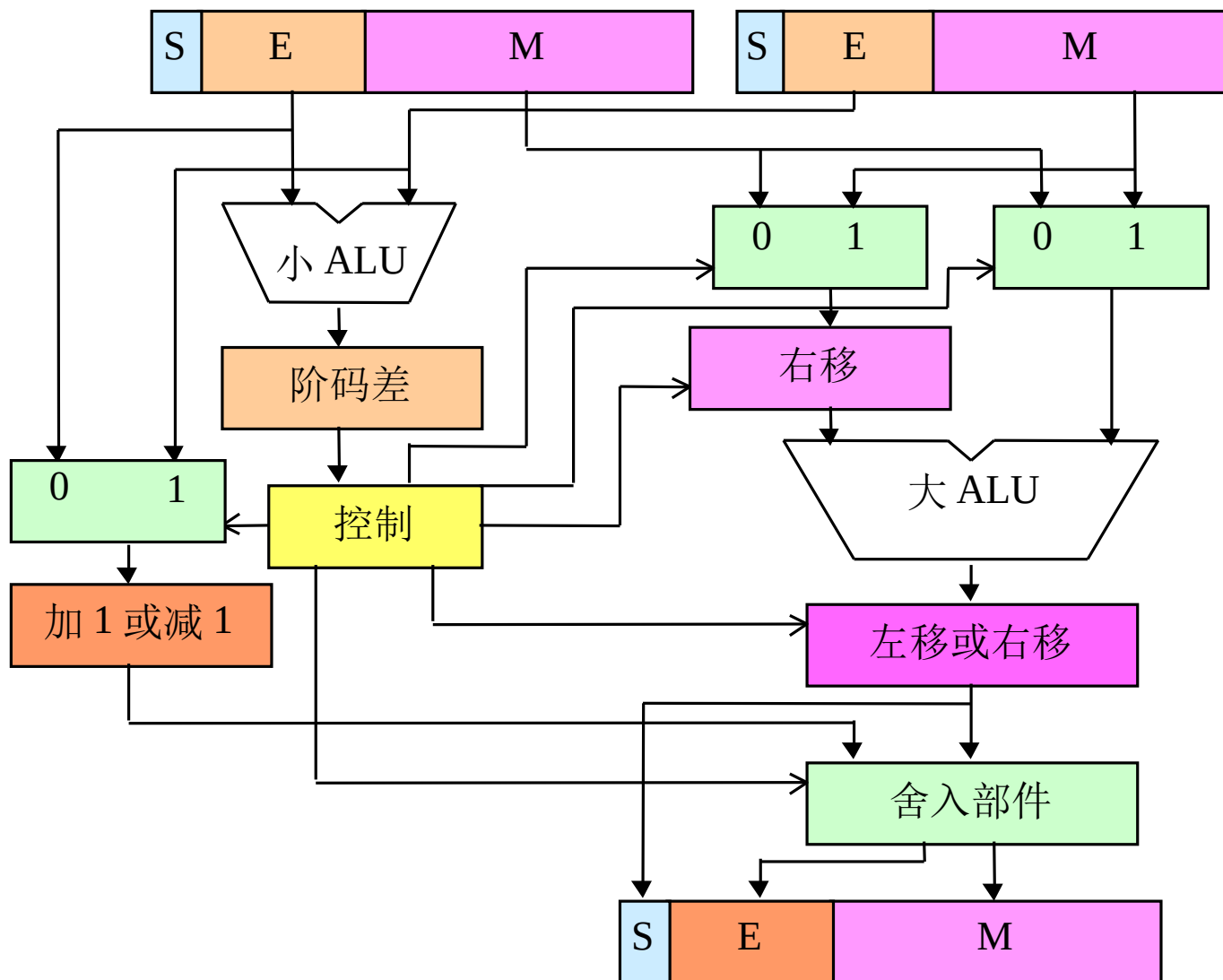
步骤4: 截去。

$$[x+y]_{\text{浮}} = 00\ 10, 11.0010$$

步骤5: 数据无溢出, 因此结果为

$$x+y = 2^{10} \times (-0.1110)$$

浮点运算电路



浮点加法器原理框图

浮点乘、除法运算

1. 浮点乘法、除法运算规则

设有两个浮点数 x 和 y :

$$x = 2^{E_x} \cdot M_x$$
$$y = 2^{E_y} \cdot M_y$$

浮点乘法运算的规则是: $x \times y = 2^{(E_x + E_y)} \cdot (M_x \times M_y)$

即: 乘积的尾数是相乘两数的尾数之积;
乘积的阶码是相乘两数的阶码之和。

浮点除法运算的规则是: $x \div y = 2^{(E_x - E_y)} \cdot (M_x \div M_y)$

即: 商的尾数是相除两数的尾数之商;
商的阶码是相除两数的阶码之差。

2. 浮点乘、除法运算步骤

浮点数的乘除运算大体分为四步：

- (1) 0 操作数检查；
- (2) 阶码加/减操作；
- (3) 尾数乘/除操作；
- (4) 结果规格化及舍入处理。

(2) 浮点数的阶码运算

- 对阶码的运算有+1、-1、两阶码求和、两阶码求差四种，运算时还必须检查结果是否溢出。
- 在计算机中，阶码通常用补码或移码形式表示。

①移码的运算规则和判定溢出的方法

移码的定义为 $[x]_{\text{移}} = 2^n + x \quad -2^n \leq x < 2^n$

按此定义，则有 $[x]_{\text{移}} + [y]_{\text{移}} = 2^n + x + 2^n + y$

$$= 2^n + (2^n + (x + y))$$

$$= 2^n + [x + y]_{\text{移}}$$

$$[x + y]_{\text{移}} = -2^n + [x]_{\text{移}} + [y]_{\text{移}}$$

②混合使用移码和补码

考虑到移码和补码的关系：

对同一个数值, 其数值位完全相同, 而符号位正好完全相反。

$[y]_{\text{补}}$ 的定义为 $[y]_{\text{补}} = 2^{n+1} + y$

则求阶码和用如下方式完成：

$$[x]_{\text{移}} + [y]_{\text{补}} = 2^n + x + 2^{n+1} + y$$

$$= 2^{n+1} + (2^n + (x + y))$$

即：

$$[x + y]_{\text{移}} = [x]_{\text{移}} + [y]_{\text{补}} \quad (\text{mod } 2^{n+1})$$

同理：

$$[x - y]_{\text{移}} = [x]_{\text{移}} + [-y]_{\text{补}} \quad (\text{mod } 2^{n+1})$$

③ 阶码运算结果溢出处理

使用双符号位的阶码加法器，并规定移码的第二个符号位，即最高符号位恒用 0 参加加减运算，则溢出条件是结果的最高符号位为1：

- 当低位符号位为 0 时，(10) 表明结果上溢，
- 当低位符号位为 1 时，(11) 表明结果下溢。
- 当最高符号位为 0 时，表明没有溢出：

 低位符号位为 1，(01) 表明结果为正；
 为 0，(00) 表明结果为负。

例： $x = +011$, $y = +110$, 求 $[x+y]_{\text{移}}$ 和 $[x-y]_{\text{移}}$, 并判断是否溢出。

解： 阶码取3位（不含符号位），其对应的真值范围是 $-8 \sim +7$

$$[x]_{\text{移}} = 01\ 011, \quad [y]_{\text{补}} = 00\ 110, \quad [-y]_{\text{补}} = 11\ 010$$

$$[x+y]_{\text{移}} = [x]_{\text{移}} + [y]_{\text{补}} =$$

$$\begin{array}{r} 01\ 011 \\ +\ 00\ 110 \\ \hline 10\ 001 \end{array}$$

结果上溢。

$$[x-y]_{\text{移}} = [x]_{\text{移}} + [-y]_{\text{补}} =$$

$$\begin{array}{r} 01\ 011 \\ +\ 11\ 010 \\ \hline 00\ 101 \end{array}$$

结果正确，为-3。

(3) 尾数处理

浮点加减法对结果的规格化及舍入处理也适用于浮点乘法。

第一种方法是：

无条件地丢掉正常尾数最低位之后的全部数值。

这种办法被称为截断处理，好处是处理简单，缺点是影响结果的精度。

第二种办法是：

运算过程中保留右移中移出的若干高位的值，最后再按某种规则用这些位上的值修正尾数。

这种处理方法被称为舍入处理。

舍入处理

- 当尾数用原码表示时:

最简便的方法是，只要尾数的最低位为1，或移出的几位中有为1的数值位，就使最低位的值为1。

另一种是0舍1入法，即当丢失的最高位的值为1时，把这个1加到最低数值位上进行修正，否则舍去丢失的各位的值。这样处理时，舍入效果对正数负数相同，入将使数的绝对值变大，舍则使数的绝对值变小。

- 当尾数是用补码表示时:

采用0舍1入法时, 若丢失的位不全为0时:

对正数来说, 舍入的结果与原码分析相同;

对负数来说, 舍入的结果与原码分析相反, 即“舍”使绝对值变大, “入”使绝对值变小; 为使原、补码舍入处理后的结果

相同, 对负数可采用如下规则进行舍入处理:

当丢失的各位均为0时, 不必舍入;

当丢失的最高位为0, 以下各位不全为0时, 或者丢失的最高位为1, 以下各位均为0时, 则舍去丢失位上的值;

当丢失的最高位为1, 以下各位不全为0时, 则执行在尾数最低位入1的修正操作。

例 设 $[x_1]_{\text{补}} = 11.01100000$, $[x_2]_{\text{补}} = 11.01100001$,
 $[x_3]_{\text{补}} = 11.01101000$, $[x_4]_{\text{补}} = 11.01111001$, 求执行只保留
小数点后4位有效数字的舍入操作值。

解： 执行舍入操作后,其结果值分别为

$$[x_1]_{\text{补}} = 11.0110 \quad (\text{不舍不入})$$

$$[x_2]_{\text{补}} = 11.0110 \quad (\text{舍})$$

$$[x_3]_{\text{补}} = 11.0110 \quad (\text{舍})$$

$$[x_4]_{\text{补}} = 11.1000 \quad (\text{入})$$

例 设有浮点数 $x = 2^{-5} \times 0.0110011$, $y = 2^3 \times (-0.1110010)$, 阶码用4位移码表示, 尾数 (含符号位)用8位补码表示。求 $[x \times y]_{\text{浮}}$ 。要求用补码完成尾数乘法运算, 运算结果尾数保留高8位(含符号位), 并用尾数低位字长值处理舍入操作。

解: 移码采用双符号位, 尾数补码采用单符号位, 则有

$$[M_x]_{\text{补}} = 0.0110011, \quad [M_y]_{\text{补}} = 1.0001110,$$

$$E_x]_{\text{移}} = 00 \ 011, \quad [E_y]_{\text{移}} = 01 \ 011, \quad [E_y]_{\text{补}} = 00 \ 011,$$

$$[x]_{\text{浮}} = 00 \ 011, 0.0110011, \quad [y]_{\text{浮}} = 01 \ 011, 1.0001110$$

(1) 求阶码和

$$[E_x + E_y]_{\text{移}} = [E_x]_{\text{移}} + [E_y]_{\text{补}} = 00\ 011 + 00\ 011 = 00\ 110,$$

值为移码形式-2。

(2) 尾数乘法运算

可采用补码阵列乘法器实现，即有

$$\begin{aligned} [M_x]_{\text{补}} \times [M_y]_{\text{补}} &= [0.0110011]_{\text{补}} \times [1.0001110]_{\text{补}} \\ &= [1.1010010, 1001010]_{\text{补}} \end{aligned}$$

(3) 规格化处理

乘积的尾数符号位与最高数值位符号相同, 不是规格化的数, 需要左规, 阶码变为00 101(-3), 尾数变为 1.0100101,0010100。

(4) 舍入处理

尾数为负数, 取尾数高位字长, 按舍入规则, 舍去低位字长, 故尾数为1.0100101。

最终相乘结果为 $[x \times y]_{\text{浮}} = 00\ 101, 1.0100101$

其真值为 $x \times y = 2^{-3} \times (-0.1011011)$

小结:

- 1) IEEE 754 浮点数的表示
- 2) 尾数的规格化表示
- 3) 浮点数的计算流程
- 4) 浮点数计算的硬件实现